

OJT Project

Student Name(s): Chinthaginjala Madhav Sai Kiran & Charan tej guniganti

Roll No(s): 25100010700016 &

Year & Section: Sem 3 A

Project Title (as assigned): DoctorDirect

Project Type: Application development

Stack / Framework: React Native, Expo, TypeScript, React Navigation, Redux Toolkit, SQLite, WebRTC, Groq API

1. Problem Understanding

1.1 What is the problem statement in your own words?

Patients often face difficulties in finding available doctors, checking appointment slots, booking appointments, and accessing consultation and prescription information in one place. DoctorDirect aims to solve this by providing a single mobile application for doctor appointment booking and online consultation, while also allowing important information to remain accessible during poor or unavailable internet connectivity. After a consultation, patients may also need a clear record of the discussion, medical recommendations, and prescribed medicines. DoctorDirect addresses this by generating an AI-assisted consultation summary that is reviewed and approved by the doctor before being shared with the patient.

1.2 Why does this problem exist or matter?

The problem matters because patients may have to use different methods to find doctors, book appointments, attend consultations, and access prescriptions. This can make the process inconvenient and time-consuming. In addition, unreliable internet connectivity can make healthcare information difficult to access. DoctorDirect addresses these issues by bringing appointment booking, tele-consultation, and prescription access into one mobile application with offline support.

1.3 Key inputs and expected outputs:

Inputs	Process	Expected Outputs
Patient Data: Login details, profile information, and patient preferences. Doctor Data: Doctor profiles, specialization, availability, and appointment slots received from the API. Appointment Data: Selected doctor, date, time slot, and booking details. Consultation Data: Video-call session and consultation information. Prescription Data: Prescription details provided by the doctor. Local Data: Previously synchronized appointments and prescriptions stored in SQLite. Summary Data: Appointment history, consultation details, prescription	Authentication: Validate patient and doctor login and identify their user role. Doctor & Slot Management: Retrieve and display available doctors and appointment slots. Appointment Processing: Validate and create appointment bookings through the API. Offline Data Handling: Store important data locally and display it when the internet is unavailable. Synchronization: Synchronize locally stored data with the server when connectivity is restored. Consultation: Establish and manage video consultation between patient and doctor.	Patient Dashboard: Upcoming appointments, recent consultation information, active prescriptions, doctor information, and consultation status. Doctor Dashboard: Upcoming appointments, patient information, and consultation schedule. Appointment Confirmation: Successfully booked appointment with date and time. Video Consultation: Patients and doctors can join an online consultation. Prescription Viewer: Patients can view prescriptions provided by the doctor. Notifications: Appointment

information, doctor information, and follow-up details used to generate patient and consultation summaries.	Notification Processing: Send appointment and consultation reminders to users. Prescription Management: Store and display prescriptions provided by doctors. Summary Generation: Combine relevant appointment, consultation, and prescription information to provide a concise patient/consultation summary.	and consultation reminders. Offline Access: Previously synchronized important data remains accessible without the internet. Appointment History: Record of completed and upcoming appointments. Patient Summary: Consolidated view of upcoming appointments, recent consultations, active prescriptions, and relevant healthcare information.
---	--	--

2. Functional Scope

2.1 What are the core features you plan to build (must-haves)?

These are the non-negotiable deliverables required for your final presentation:

- **Patient & Doctor Authentication** – Separate login and role-based dashboards for patients and doctors.
- **Doctor Search & Profiles** – Patients can browse doctors, view specialization, and check availability.
- **Appointment & Slot Booking** – Patients can select available dates and time slots and book appointments.
- **Doctor Appointment Management** – Doctors can view upcoming appointments, patient details, and consultation schedules.
- **Appointment Rescheduling & Cancellation** – Allow patients to modify or cancel

appointments based on availability.

- **Video Tele-Consultation** – Patients and doctors can join scheduled online consultations through video calls.
- **Prescription Management** – Doctors can add prescriptions, while patients can view their prescribed medicines and instructions.
- **Offline Data Access** – Previously synchronized appointments and prescriptions remain accessible without an internet connection.
- **API Integration & Data Synchronization** – Connect the app to the backend and synchronize local data when the internet is restored.
- **Push Notifications** – Send appointment confirmations, reminders, and consultation notifications to patients and doctors.

2.2 What stretch goals could you attempt if time permits?

To make this project "production-ready" or more advanced for your portfolio:

- **Dark Mode** – Add a theme toggle so users can switch between light and dark interfaces.
- **Advanced Push Notifications** – Provide reminders for upcoming appointments, consultation start times, and prescription updates.
- **Doctor Availability Management** – Allow doctors to add, update, and block their available appointment slots.
- **Appointment History & Search** – Provide patients and doctors with searchable records of past consultations and appointments.
- **Profile & Settings** – Add profile editing, notification preferences, and other user settings.
- **Enhanced Offline Synchronization** – Improve data synchronization and handle pending updates when connectivity is restored.
- **EAS Build & Deployment Automation** – Configure automated Android/iOS builds for easier application delivery.
- **Automated Testing** – Expand Jest test coverage for critical application features.

2.3 Which libraries or tools will you use?

React Native – Cross-platform mobile application framework

Expo – Development and application tooling

TypeScript – Application programming language

React Navigation – Screen and navigation management

Redux Toolkit – Application state management

Expo SQLite – Local database and offline data storage

WebRTC – Real-time video consultation

Axios – REST API communication

3. System & Design Thinking

3.1 Sketch or describe your app flow / pipeline:

**Start App → User Login/Registration → Identify Patient/Doctor Role → Load
Dashboard & Summary → Fetch Data from API → Store Relevant Data Locally
in SQLite → Display Doctors/Appointments → Book or Manage Appointment
→ Send Notification → Join Video Consultation → Create/View Prescription
→ View Consultation/Patient Summary → Sync Local Data with Server When
Online**

3.2 What data structures or algorithms are central to this project?

SQLite Tables → Store users, doctors, appointments, slots, and prescriptions locally.

State Management → Manage authentication, user roles, appointments, and application state.

Appointment Validation → Validate doctor availability and selected time slots before

booking.

Offline Synchronization → Synchronize locally stored data with the server when internet connectivity is restored.

Role-Based Access Logic → Provide different features and dashboards for patients and doctors.

API Data Processing → Fetch, validate, and process data received from the backend APIs.

Video Session Management → Manage video consultation sessions and connection states using WebRTC.

3.3 How will you test correctness or performance?

- **Functional Testing** → Verify login, doctor search, appointment booking, slot selection, prescriptions, notifications, and video consultation.
- **Offline Testing** → Disable internet and verify that previously synchronized appointments and prescriptions remain accessible.
- **Synchronization Testing** → Restore internet and verify that local data synchronizes correctly with the server.
- **API Testing** → Verify that API requests and responses are handled correctly.
- **Role-Based Testing** → Ensure patients and doctors can access only their respective features and dashboards.
- **Video Call Testing** → Test video consultation connection, joining, and disconnection scenarios.
- **Unit Testing** → Use Jest to test important application logic and maintain adequate test coverage.
- **Performance Testing** → Check screen loading, navigation, scrolling, and overall responsiveness to maintain smooth app performance.
- **Summary Testing** → Verify that patient and consultation summaries correctly display relevant appointment, consultation, prescription, and doctor information.

4. Timeline & Milestones (9 Weeks)

Week	Planned Deliverables	Mentor Checkpoint
W1	Project scope, requirements, user roles, and technology stack	
W2	UI wireframes, screen designs, and patient/doctor user flows	
W3	App architecture, navigation structure, database design, and state management setup	
W4	Authentication, role-based dashboards, and core navigation implementation	
W5	Doctor profiles, availability, appointment slots, and booking functionality	
W6	REST API integration, SQLite offline storage, and data synchronization	
W7	Video consultation, prescription management, and push notifications	
W8	Integration testing, offline testing, error handling, and bug fixing	
W9	Performance optimization, Jest testing, dark mode, and EAS build configuration	
W10	Final integration, documentation, demo preparation, and viva	

5.Risks & Dependencies

5.1 What's the hardest part technically for you right now?

- Implementing reliable role-based authentication for separate patient and doctor workflows.
- Managing appointment and slot availability correctly between patients and doctors.
- Implementing offline-first data storage and synchronization using SQLite.
- Handling API failures, network loss, and synchronization conflicts reliably.
- Integrating and managing real-time video consultation using WebRTC.
- Ensuring secure handling of patient and prescription data across the app and backend.

5.2 What dependencies or help do you need from mentors?

- Guidance on application architecture and selection of appropriate technologies.
- Advice on database design for users, doctors, appointments, slots, and prescriptions.
- Guidance on offline-first implementation and SQLite data synchronization.
- Support with video consultation integration and WebRTC implementation.
- Feedback on API design, authentication, and role-based access.
- Review of testing strategy, performance, and overall application reliability.
- Feedback on final integration, documentation, and demonstration.

6.Evaluation Readiness

6.1 How will you prove that your project “works”?

- **End-to-End Demo** → Demonstrate the complete flow from login to appointment booking, consultation, and prescription viewing.
- **Patient & Doctor Testing** → Show separate workflows and dashboards for both user roles.
- **Appointment Testing** → Demonstrate slot selection, booking, confirmation, and doctor-side appointment management.
- **Video Consultation Testing** → Demonstrate a patient and doctor joining the same video consultation.
- **Offline Testing** → Disable internet and verify that previously synchronized appointments and prescriptions remain accessible.
- **Synchronization Testing** → Restore the internet and verify that local data synchronizes correctly with the server.
- **API Testing** → Verify successful communication between the mobile application and backend.
- **Automated Testing** → Use Jest to test important application logic and document test results.
- **Performance Testing** → Verify smooth navigation, screen loading, and overall application responsiveness.
- **Final Build Demonstration** → Provide a working Android/iOS build with the implemented core features.

6.2 What success metric or goal will you aim for?

- **Core Feature Completion** → Successfully implement and demonstrate all must-have features.
- **End-to-End Workflow** → Complete patient booking → doctor confirmation → video consultation → prescription flow without critical errors.
- **Offline Reliability** → Ensure previously synchronized appointments and prescriptions remain accessible during network loss.
- **Data Synchronization** → Ensure local data synchronizes correctly with the server after connectivity is restored.
- **Application Performance** → Maintain smooth navigation, screen transitions, and responsive UI during normal usage.
- **Testing** → Achieve 80%+ test coverage for critical application logic.

- **Stability** → Complete the final demonstration without any critical application crashes or blocking defects.

7.Responsibilities

7.1 Responsibilities

Task	Student 1	Student 2	Mentor Notes
------	-----------	-----------	--------------

Authentication & Role Management: Implement patient/doctor login, registration, and role-based access.	☐	☐	
App Development: Develop UI screens, navigation, patient/doctor dashboards, and user flows.	☐	☐	
Appointment Management: Implement doctor availability, slot selection, appointment booking, and appointment management.	☐	☐	

Backend & API Integration: Implement REST API integration, data handling, and synchronization between the app and backend.	☐	☐	
Offline Storage: Implement SQLite storage, offline data access, and synchronization when connectivity is restored.	☐	☐	
Video & Prescription Features: Implement video consultation and prescription creation/viewing functionality.	☐	☐	
Testing & Final Delivery: Test features, fix bugs, optimize performance, document the project, and prepare the final demo.	☐	☐	

Signatures (Students):



Mentor Approval: Kshitiz Dhooria Sir

Date: 27 /8/26